

# HW4: LLM Tool Calling Agent

張家睿 111550132

## Q1. Method Description (10%)

### 任務目標

對於 `test.jsonl` 中每一筆 (`id`, `full_context`, `current_step`, `options`)，從候選工具 A–H 中選出單一字母作為 `current_step` 對應的正確工具。輸出 CSV 兩欄 `id`, `answer`。Metric 為 `categorization accuracy`。

### Overall Pipeline (Prompt-based, 主力提交版本)

在 `HW4_111550132.py`，使用 `google/gemma-4-31b-it` (NVIDIA NIM 的 API) 作為推論後端。Pipeline 設計：

```
test.jsonl → prompt builder → LLM (gemma-4-31b-it) → answer parser (multi-tier regex)
→ resume-safe writer → submission.csv
```

關鍵模組：

- Prompt builder** (`build_prompt`)：把 `full_context`, `current_step` 與 `options` 中每個工具的 (`name`, `description`, `arguments.properties`, `results.properties`) 組成單一 user message，並在尾段明確要求模型在獨立一行輸出 `Answer: <LETTER>`。Prompt 內也列出歧義例 (`search_train` vs `query_past_ticket`) 作為 disambiguation 提示。
- Multi-tier answer parser** (`extract_answer_letter`)：以五級回退規則從模型回應抽出唯一字母——`Answer: X 行` → `<answer>X</answer>` → `**X**` → `X` → 取最後一個出現在 `valid_letters` 內的 standalone letter。確保即使模型寫了一整段散文也能穩定抽到答案。

### 本地推論版 (供 Q2 / Q3 同類模型對照)

`local_llm_inference.py` 把同一條 pipeline 換到 `Qwen3.6-27B-Q4_K_M GGUF` + `llama-cpp-python`。

Prompt + parser 與 `HW4_111550132.py` 完全對齊，方便 Q2 比較。

### Kaggle 結果

| 提交 | 模型                                      | Public LB      | 備註                               |
|----|-----------------------------------------|----------------|----------------------------------|
| 1  | <code>gemma-4-31b-it</code> (NIM)       | <b>0.90112</b> | Prompt + Full-Info, thinking off |
| 2  | <code>Qwen3.6-27B Q4_K_M</code> (local) | 0.87005        | Prompt + Full-Info, thinking off |

## Q2. Comparison of Methods (10%)

### Experimental Setup

Prompt 與 SFT 兩條 pipeline 跑在 **同一顆模型** 上。考慮：

- 主提交用的 `gemma-4-31b-it` 透過 NIM API 提供，**無法做 fine-tune**；
- 使用 `Qwen3.6-27B` **做 QLoRA fine-tune**。

所有實驗的 dev set 為從 `train.jsonl` 取最後 1000 筆 (`build_or_load_dev_split`，由 `data/dev.jsonl` 落盤，兩種 prompt mode 共用同一份 dev 確保可比性)。SFT 訓練資料則排除這 1000 筆 dev id，避免污染評估。

兩種 prompt configuration 定義：

| Configuration   | 提供給 model 的工具資訊                                                                                                                 |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------|
| Full-Info       | tool.name + tool.description + 每個 argument 的 (key, type, description) + 每個 result 的 (key, type, description)                    |
| Structural-Only | 僅 每個 argument 的 (key, type) + 每個 result 的 (key, type)；完全剔除：tool name、tool description、argument descriptions、result descriptions |

實作上，run\_dev\_eval.py 內的 `_format_tool_full` / `_format_tool_struct` 兩個函式對應，並由同一份 `extract_answer_letter` 抽答案；evaluation + Q3 錯誤分類 (`evaluate_and_analyze`) 完全共用。

## 結果總表

| Approach     | Configuration   | Dev Accuracy            | # Errors | Ambiguity err. (%) |
|--------------|-----------------|-------------------------|----------|--------------------|
| Prompt-based | Full-Info       | <b>0.9760</b> (975/999) | 24       | 11 / 24 = 45.8%    |
| Prompt-based | Structural-Only | 0.9119 (911/999)        | 88       | 48 / 88 = 54.5%    |
| SFT (QLoRA)  | Full-Info       | (訓練中)                   | TBA      | TBA                |
| SFT (QLoRA)  | Structural-Only | (訓練中)                   | TBA      | TBA                |

SFT 設定：QLoRA on Qwen3.6-27B 4-bit (NF4 + double quant)，LoRA  $r=8$  /  $\alpha=16$  掛在 attention (q,k,v,o)，`max_seq_len=1024`，train data 3000 筆 (從 `train.jsonl` 去 dev 後隨機抽樣)，1 epoch、`batch_size=1`、`grad_accum=4`、`paged_adamw_8bit`、bf16 + gradient checkpointing。在 dual 4090 上以 `CUDA_VISIBLE_DEVICES` 各鎖一張卡平行訓練，預計 3 小時。

## Prompt-based: Full-Info vs Structural-Only

Full-Info 勝 (97.60% vs 91.19%，差距 6.4 pp)。可能原因：

1. **Tool name 本身就是強訊號**。Spec 給的工具命名是 verb-noun 結構 (例：`train_ticket_booking`，`search_accommodation`)，model 可以直接以 `current_step` 的動詞 (book / search / query / cancel) 與 tool name 做表面 string match。一旦把 name 拔掉，這條捷徑就斷了。
2. **Description 是 argument key 的「中介編碼」**。例如 argument key `seatType` 在 Full-Info 下被 description “Seat type (Hard sleeper / Soft sleeper)” 強化語意，Structural-Only 下只剩裸鍵名，model 必須完全靠鍵名 + 上下文推論其用途。雖然鍵名命名規則 (snake\_case / camelCase) 通常還能讓 model 部分解讀，但失去描述會在 **鍵名相似但語意不同** 的工具上產生混淆。
3. **Ambiguity 錯誤比例上升**。Full 模式 11/24=45.8% 是 ambiguity 錯誤，Struct 模式跳升到 48/88=54.5%。也就是說，Struct 額外多出來的 64 個錯誤中，有約 (48-11)/64≈58% 來自工具名/描述被拿掉後 model 把語意相近的工具混在一起 (詳見 Q3)。

## SFT: Full-Info vs Structural-Only

(訓練尚未結束，結果待補。)

預期方向：SFT 訓練後兩配置的差距會 **縮小**，因為微調讓 model 在 train 分布內直接學到 schema → 答案的映射，descriptions 帶來的「語意捷徑」不再是唯一資訊管道；不過 Structural-Only 仍可能略輸，因為它在分佈外 (e.g. test set 中是 train 沒見過的工具) 缺乏 descriptions 來 ground 鍵名。

## Q3. Tool Ambiguity and Misselection (10%)

### 處理 Ambiguity 的策略

主 pipeline 同時用了三層手段：

1. **Prompt 中明確提點歧義範例**。build\_prompt 的 reasoning guidelines 直接寫：

Two tools may have similar names (e.g. `search\_train` vs `query\_past\_ticket`); disambiguate by carefully comparing each tool's argument keys and result keys against what the current step actually requires.

把真實歧義對直接放進 prompt，引導 model 不要只看 tool name 表面。

2. **把工具的 schema 攤平丟給 model**。除了 name + description，我們把 arguments.properties 與 results.properties 的 **每個 (key, type, description)** 都列在 prompt 內，等於把 OpenAPI-style schema 直接餵給模型，讓它能用「需要哪些輸入 / 會產生哪些輸出」這兩個鈎子做二次驗證，而不只是看名字。
3. **Multi-tier answer parser + retry**。即使 model 在 ambiguity 下吐出像 “Maybe G but probably B” 這種猶豫式答案，五級 fallback 解析也會擇一合法 letter；若整段回應失敗 (timeout / 空字串)，則重打到指定次數。

實驗顯示，這套組合在 Full-Info 設定下把 dev ambiguity 錯誤壓到 11 題 / 999 (1.1%)。

## 量化錯誤統計 (Quantitative Error Stats)

評估方式：將模型在 dev set (1000 筆) 上的預測 vs gold 比對，對每一筆錯誤計算

$$\text{Jaccard}(t_{\text{pred}}, t_{\text{gold}}) = \frac{|t_{\text{pred}} \cap t_{\text{gold}}|}{|t_{\text{pred}} \cup t_{\text{gold}}|}$$

其中  $t_{\text{pred}} / t_{\text{gold}}$  是把預測 / 正解工具的 name 以 `[_\-\s]+` tokenize 後的詞袋 (snake\_case → token set)。  
若  $\text{Jaccard} \geq 0.4$ ，該錯誤判定為 **tool ambiguity error**。閾值 0.4 對應「兩工具共享  $\geq 40\%$  的 token」，正好覆蓋 spec 給的範例：

| Pair                                      |    | Tokens                                                                  | Jaccard | Classified as ambiguity? |
|-------------------------------------------|----|-------------------------------------------------------------------------|---------|--------------------------|
| train_ticket_query<br>search_train        | vs | $\{\text{train, ticket, query}\} \cap \{\text{train, search}\}$         | 0.25    | ×                        |
| train_ticket_query<br>train_ticket_search | vs | $\{\text{train, ticket, query}\} \cap \{\text{train, ticket, search}\}$ | 0.50    | ✓                        |
| book_flight<br>vs cancel_flight           |    | $\{\text{book, flight}\} \cap \{\text{cancel, flight}\}$                | 0.33    | ×                        |
| login_to_ticket_platform<br>vs logout     |    | $\{\text{login, to, ticket, platform}\} \cap \{\text{logout}\}$         | 0.00    | ×                        |

實際統計結果：

| Approach             | # samples | # correct | # errors | # ambiguity | % of errors |
|----------------------|-----------|-----------|----------|-------------|-------------|
| Prompt + Full-Info   | 999       | 975       | 24       | 11          | 45.8%       |
| Prompt + Struct-Only | 999       | 911       | 88       | 48          | 54.5%       |
| SFT + Full-Info      | TBA       | TBA       | TBA      | TBA         | TBA         |
| SFT + Struct-Only    | TBA       | TBA       | TBA      | TBA         | TBA         |

## 觀察

1. **接近一半的錯誤源自 ambiguity**。即使在表現最好的 Prompt + Full-Info 上，仍有 45.8% 的錯題是「預測了 token-相似  $\geq 40\%$  的工具」造成的。也就是 model 並非什麼都做不到，而是在「兩個名字相近、做的事情不同」這類細節上會卡住。
2. **拿掉 descriptions 會放大 ambiguity 比例**。從 45.8% 升到 54.5%，符合直覺：當 model 無法靠描述 (e.g. “Used for booking” vs “Used for cancellation”) disambiguate 時，會更依賴 name 的字面相似度，這正好就是 ambiguity 錯誤的定義。

3. **主要混淆對是  $A \leftrightarrow B$** 。在兩種設定下 top confusion 都是  $B \rightarrow A$  與  $A \rightarrow B$ ，反映 train.jsonl 的選項分布 (大多只給 A-D)，且 A、B 往往是同一家族 (e.g. search\_train 與 train\_ticket\_query) 的兩個變體。

### 後續可嘗試的改進

- **Few-shot prompt augmentation**：從 train.jsonl 抽出 ambiguity 配對較多的樣本當作 in-context exemplar。
- **Self-consistency**：對同一筆樣本用較高 temperature 跑數次，多數決抽 letter，預期可進一步降 ambiguity error。
- **SFT 後 inference**：若 SFT 跑完 dev accuracy 提升，表示 model 已把 spec 中的 ambiguity 對映關係內化，prompt 可以更短。

## Appendix: Code Layout

| 檔案                     | 用途                                                                                                  |
|------------------------|-----------------------------------------------------------------------------------------------------|
| HW4_111550132.py       | Kaggle 主提交 pipeline (NIM API + gemma-4-31b-it)，多 API key + worker pool                              |
| local_llm_inference.py | 本地 GGUF 推論 (llama-cpp-python)，單卡 / 雙卡平行版本                                                           |
| run_dev_eval.py        | Dev set 評估腳本，支援 <code>--prompt-mode {full, struct}</code> 兩種 configuration + Q3 ambiguity 錯誤分類      |
| train_sft.py           | QLoRA fine-tuning on Qwen3.6-27B (4-bit base + LoRA on attention)，對應 Q2 SFT $\times$ {Full, Struct} |
| eval_sft.py            | SFT 後評估腳本 (transformers + peft adapter)，沿用 run_dev_eval 的 prompt 與 evaluation 邏輯                    |
| data/dev.jsonl         | 從 train.jsonl 取最後 1000 筆，所有 dev 實驗共用同一份以確保 4-cell 可比性                                               |